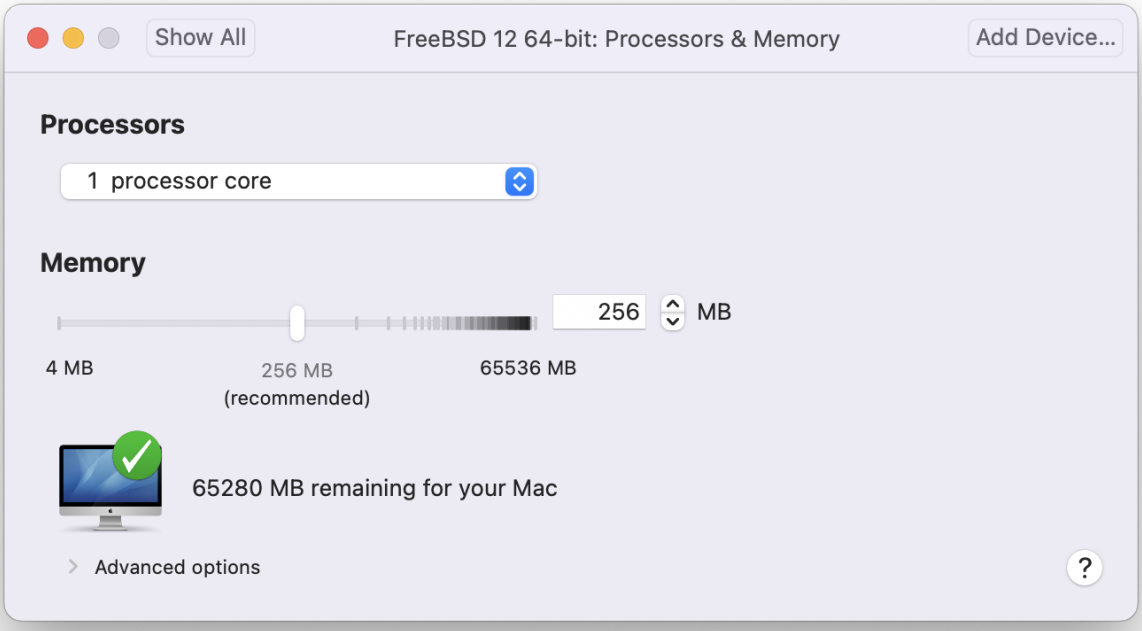
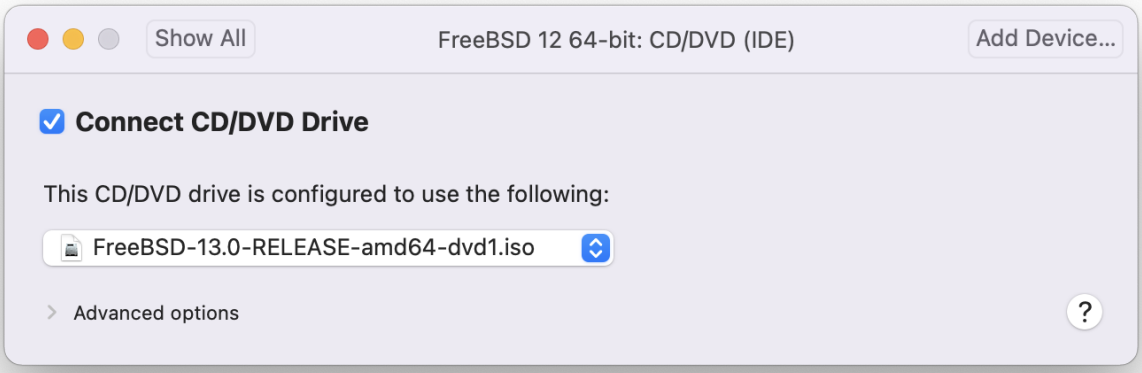
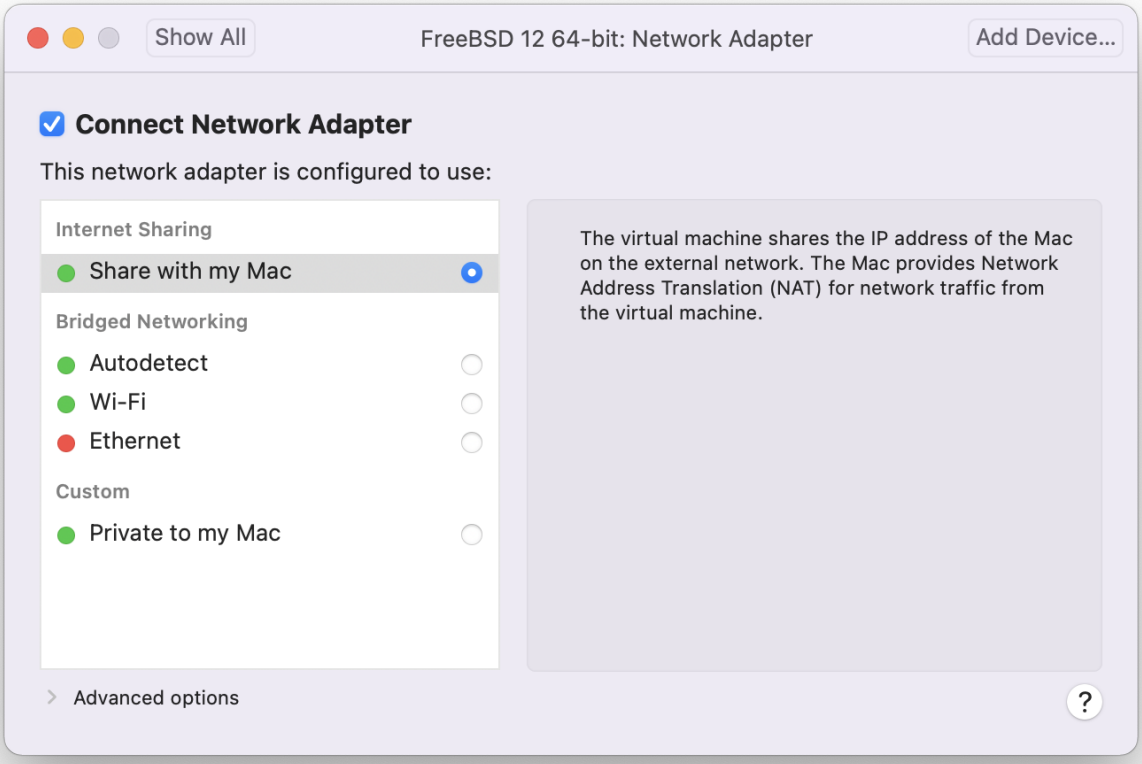


The status of the CD-ROM device. Normally the CD/DVD/ISO is disconnected from the virtual machine when it is no longer needed.



The last thing to change is how the virtual machine will connect to the network. To allow connections to the virtual machine from other machines besides the host, choose Connect directly to the physical network (Bridged). Otherwise, Share the host's internet connection (NAT) is preferred so that the virtual machine can have access to the Internet, but the network cannot access the virtual machine.



After modifying the settings, boot the newly installed FreeBSD virtual machine.

24.3.2. Configuring FreeBSD on VMware Fusion

After FreeBSD has been successfully installed on macOS® X with VMware Fusion, there are a number of configuration steps that can be taken to optimize the system for virtualized operation.

1. Set Boot Loader Variables

   The most important step is to reduce the `kern.hz` tunable to reduce the CPU utilization of FreeBSD under the VMware Fusion environment. This is accomplished by adding the following line to /boot/loader.conf:

   ```
   kern.hz=100
   ```

   Without this setting, an idle FreeBSD VMware Fusion guest will use roughly 15% of the CPU of a single processor iMac®. After this change, the usage will be closer to 5%.

2. Create a New Kernel Configuration File

   All FireWire, and USB device drivers can be removed from a custom kernel configuration file. VMware Fusion provides a virtual network adapter used by the em(4) driver, so all network devices except for em(4) can be removed from the kernel.

3. Configure Networking

The most basic networking setup uses DHCP to connect the virtual machine to the same local area network as the host Mac®. This can be accomplished by adding `ifconfig_em0="DHCP"` to /etc/rc.conf. More advanced networking setups are described in Advanced Networking.

4. Install drivers and open-vm-tools

   To run FreeBSD smoothly on VMWare, drivers should be installed:

   ```
   # pkg install xf86-video-vmware xf86-input-vmmouse open-vm-tools
   ```

   The xf86 packages are only available for x86 FreeBSD guests.

Enabling the mouse

+ Some users have reported issues using the mouse in the virtual machine. The mouse may be enabled by appending the following to /boot/loader.conf:

+

```
# ums_load="YES"
```

24.4. FreeBSD as a Guest on VirtualBox™

FreeBSD works well as a guest in VirtualBox™. The virtualization software is available for most common operating systems, including FreeBSD itself.

The VirtualBox™ guest additions provide support for:

- Clipboard sharing.
- Mouse pointer integration.
- Host time synchronization.
- Window scaling.
- Seamless mode.



These commands are run in the FreeBSD guest.

First, install the emulators/virtualbox-ose-additions package or port in the FreeBSD guest. This will install the port:

```
# cd /usr/ports/emulators/virtualbox-ose-additions && make install clean
```

Add these lines to /etc/rc.conf:

```
vboxguest_enable="YES"
vboxservice_enable="YES"
```

If ntpd(8) or ntpdate(8) is used, disable host time synchronization:

```
vboxservice_flags="--disable-timesync"
```

Xorg will automatically recognize the `vboxvideo` driver. It can also be manually entered in /etc/X11/xorg.conf:

```
Section "Device"
    Identifier "Card0"
    Driver "vboxvideo"
    VendorName "InnoTek Systemberatung GmbH"
    BoardName "VirtualBox Graphics Adapter"
EndSection
```

To use the `vboxmouse` driver, adjust the mouse section in /etc/X11/xorg.conf:

```
Section "InputDevice"
    Identifier "Mouse0"
    Driver "vboxmouse"
EndSection
```

Shared folders for file transfers between host and VM are accessible by mounting them using `mount_vboxvfs`. A shared folder can be created on the host using the VirtualBox GUI or via `vboxmanage`. For example, to create a shared folder called *myshare* under /mnt/bsdboxshare for the VM named *BSDBox*, run:

```
# vboxmanage sharedfolder add 'BSDBox' --name myshare --hostpath /mnt/bsdboxshare
```

Note that the shared folder name must not contain spaces. Mount the shared folder from within the guest system like this:

```
# mount_vboxvfs -w myshare /mnt
```

24.5. FreeBSD as a Host with VirtualBox™

VirtualBox™ is an actively developed, complete virtualization package, that is available for most operating systems including Windows®, macOS®, Linux® and FreeBSD. It is equally capable of running Windows® or UNIX®-like guests. It is released as open source software, but with closed-source components available in a separate extension pack. These components include support for

USB 2.0 devices. More information may be found on the Downloads page of the VirtualBox™ wiki. Currently, these extensions are not available for FreeBSD.

24.5.1. Installing VirtualBox™

VirtualBox™ is available as a FreeBSD package or port in emulators/virtualbox-ose. The port can be installed using these commands:

```
# cd /usr/ports/emulators/virtualbox-ose
# make install clean
```

One useful option in the port's configuration menu is the `GuestAdditions` suite of programs. These provide a number of useful features in guest operating systems, like mouse pointer integration (allowing the mouse to be shared between host and guest without the need to press a special keyboard shortcut to switch) and faster video rendering, especially in Windows® guests. The guest additions are available in the **Devices** menu, after the installation of the guest is finished.

A few configuration changes are needed before VirtualBox™ is started for the first time. The port installs a kernel module in /boot/modules which must be loaded into the running kernel:

```
# kldload vboxdrv
```

To ensure the module is always loaded after a reboot, add this line to /boot/loader.conf:

```
vboxdrv_load="YES"
```

To use the kernel modules that allow bridged or host-only networking, add this line to /etc/rc.conf and reboot the computer:

```
vboxnet_enable="YES"
```

The `vboxusers` group is created during installation of VirtualBox™. All users that need access to VirtualBox™ will have to be added as members of this group. `pw` can be used to add new members:

```
# pw groupmod vboxusers -m username
```

The default permissions for /dev/vboxnetctl are restrictive and need to be changed for bridged networking:

```
# chown root:vboxusers /dev/vboxnetctl
# chmod 0660 /dev/vboxnetctl
```

To make this permissions change permanent, add these lines to /etc/devfs.conf:

```
own     vboxnetctl root:vboxusers
perm    vboxnetctl 0660
```

To launch VirtualBox™, type from an Xorg session:

```
% VirtualBox
```

For more information on configuring and using VirtualBox™, refer to the official website. For FreeBSD-specific information and troubleshooting instructions, refer to the relevant page in the FreeBSD wiki.

24.5.2. VirtualBox™ USB Support

VirtualBox™ can be configured to pass USB devices through to the guest operating system. The host controller of the OSE version is limited to emulating USB 1.1 devices until the extension pack supporting USB 2.0 and 3.0 devices becomes available on FreeBSD.

For VirtualBox™ to be aware of USB devices attached to the machine, the user needs to be a member of the `operator` group.

```
# pw groupmod operator -m username
```

Then, add the following to /etc/devfs.rules, or create this file if it does not exist yet:

```
[system=10]
add path 'usb/*' mode 0660 group operator
```

To load these new rules, add the following to /etc/rc.conf:

```
devfs_system_ruleset="system"
```

Then, restart devfs:

```
# service devfs restart
```

Restart the login session and VirtualBox™ for these changes to take effect, and create USB filters as necessary.

24.5.3. VirtualBox™ Host DVD/CD Access

Access to the host DVD/CD drives from guests is achieved through the sharing of the physical drives. Within VirtualBox™, this is set up from the Storage window in the Settings of the virtual machine. If needed, create an empty IDECD/DVD device first. Then choose the Host Drive from the popup menu

for the virtual CD/DVD drive selection. A checkbox labeled Passthrough will appear. This allows the virtual machine to use the hardware directly. For example, audio CDs or the burner will only function if this option is selected.

In order for users to be able to use VirtualBox™DVD/CD functions, they need access to /dev/xpt0, /dev/cdN, and /dev/passN. This is usually achieved by making the user a member of operator. Permissions to these devices have to be corrected by adding these lines to /etc/devfs.conf:

```
perm cd* 0660
perm xpt0 0660
perm pass* 0660
```

```
# service devfs restart
```

24.6. Virtualization with QEMU on FreeBSD

QEMU is a generic machine emulator and virtualizer that is completely open source software. It is developed by a large, active community and provides support for FreeBSD, OpenBSD, and NetBSD as well as other operating systems.

From the QEMU documentation:

- QEMU can be used in several different ways. The most common is for System Emulation, where it provides a virtual model of an entire machine (CPU, memory, and emulated devices) to run a guest OS. In this mode the CPU may be fully emulated, or it may work with a hypervisor such as KVM, Xen or Hypervisor.Framework to allow the guest to run directly on the host CPU.
- The second supported way to use QEMU is User Mode Emulation, where QEMU can launch processes compiled for one CPU on another CPU. In this mode the CPU is always emulated.
- QEMU also provides a number of standalone command line utilities, such as the qemu-img(1) disk image utility that allows one to create, convert, and modify disk images.

QEMU can emulate a wide number of architectures including Arm™, i386, x86_64, MIPS™, s390X, SPARC™ (Sparc™ and Sparc64™), and others. The list of QEMU System Emulator Targets is regularly kept up to date.

This section describes how to use QEMU for both System Emulation and User Mode Emulation on FreeBSD, and provides examples of using QMEU commands and command line utilities.

24.6.1. Installing QEMU Software

QEMU is available as a FreeBSD package or as a port in emulators/qemu. The package build includes sane options and defaults for most users and is the recommended method of installation.

```
# pkg install qemu
```

The package installation includes several dependencies. Once the installation is complete, create a link to the host version of QEMU that will be used most often. If the host is an Intel™ or AMD™ 64 bit system that will be:

```
# ln -s /usr/local/bin/qemu-system-x86_64 /usr/local/bin/qemu
```

Test the installation by running the following command as a non-root user:

```
% qemu
```

This brings up a window with QEMU actively trying to boot from hard disk, floppy disk, DVD/CD, and PXE. Nothing has been set up yet, so the command will produce several errors and end with "No bootable device" as shown in Figure 1. However, it does show that the QEMU software has been installed correctly.

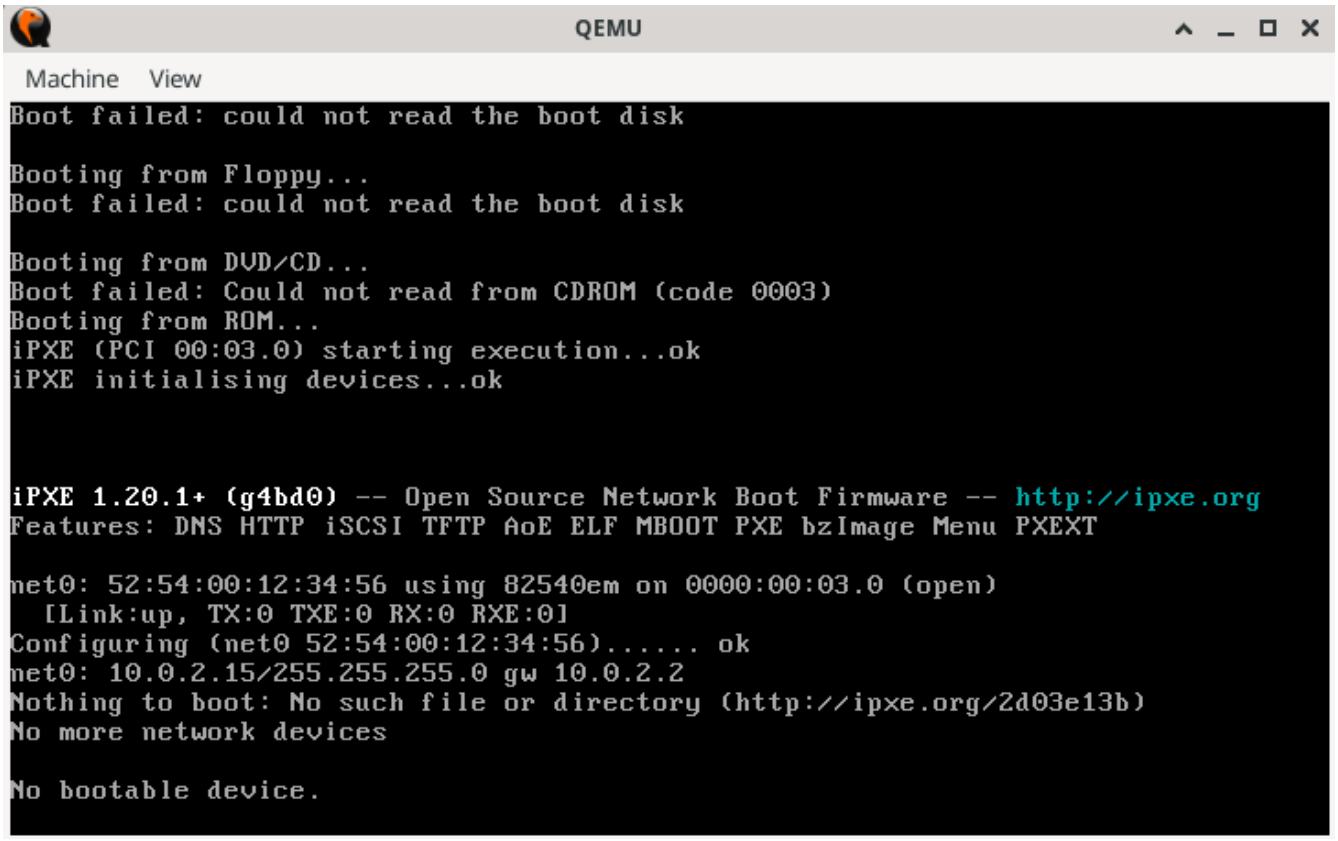


Figure 61. QEMU with no bootable image

24.6.2. Virtual Machine Install



QEMU is under very active development. Features and command options can change from one version to the next. This section provides examples developed with QEMU version 9.0.1 (Summer, 2024). When in doubt, always consult the QEMU Documentation particularly the About QEMU page which has links to supported build platforms, emulation, deprecated features, and removed features.

Follow the steps below to create two virtual machines named "**left**", and "**right**". Most commands

can be performed without root privileges.

1. Create a test environment to work with QEMU:

   ```
   % mkdir -p ~/QEMU  ~/QEMU/SCRIPTS  ~/QEMU/ISO  ~/QEMU/VM
   ```

   The SCRIPTS directory is for startup scripts and utilities. The ISO directory is for the guest ISO boot images. The VM directory is where the virtual machine images (VMs) will reside.

2. Download a recent copy of FreeBSD into ~/QEMU/ISO:

   ```
   % cd ~/QEMU/ISO
   % fetch https://download.freebsd.org/releases/ISO-IMAGES/14.1/FreeBSD-14.1-RELEASE-
   amd64-bootonly.iso
   ```

   Once the download is complete create a shorthand link. This shorthand link is used in the startup scripts below.

   ```
   % ln -s FreeBSD-14.1-RELEASE-amd64-bootonly.iso  fbsd.iso
   ```

3. Change directory to the location for virtual machines (~/QEMU/VM). Run qemu-img(1) to create the disk images for the “left” VM:

   ```
   % cd ~/QEMU/VM
   % qemu-img create -f raw  left.img   15G
   ```

   The QEMU raw format is designed for performance. The format is straightforward and has no overhead which makes it faster, especially for high performance or high throughput scenarios. The use case is for maximum performance where no additional features, such as snapshots, are needed. This format is used in the script for the "left" VM below.

   A separate format is qcow2 which uses QEMU’s "copy on write" technique for managing disk space. This technique does not require a complete 15G disk, just a stub version that is managed directly by the VM. The disk grows dynamically as the VM writes to it. This format supports snapshots, compression, and encryption. The use case for this format is for development, testing, and scenarios with the need of these advanced features. This format is used in the script for the "right" VM below.

   Run qemu-img(1) again to create the disk image for the "right" VM using qcow2:

   ```
   % qemu-img create -f qcow2 -o preallocation=full,cluster_size=512K,lazy_refcounts
   =on right.qcow2 20G
   ```

   To see the actual size of the file use:

```
% du -Ah right.qcow2
```

4. Set up networking for both virtual machines with the following commands. In this example the host network interface is `em0`. If necessary, change it to fit the interface for the host system. This must be done after every host machine restart to enable the QEMU guest VMs to communicate.

```
# ifconfig tap0 create
# ifconfig tap1 create
# sysctl net.link.tap.up_on_open=1
net.link.tap.up_on_open: 0 -> 1
# sysctl net.link.tap.user_open=1
net.link.tap.user_open: 0 -> 1
# ifconfig bridge0 create
# ifconfig bridge0 addm tap0 addm tap1 addm em0
# ifconfig bridge0 up
```

   The above commands create two tap(4) devices (`tap0`, `tap1`) and one if_bridge(4) device (`bridge0`). Then, they add the `tap` devices and the local host interface (`em0`) to the `bridge`, and set two sysctl(8) entries to allow for normal users to open the tap device. These commands will allow the virtual machines to talk to the network stack on the host.

5. Change to ~/QEMU/SCRIPTS, use the following script to start the first virtual machine, "left". This script uses the QEMU raw disk format.

```
/usr/local/bin/qemu-system-x86_64  -monitor none \
  -cpu qemu64 \
  -vga std \
  -m 4096 \
  -smp 4   \
  -cdrom ../ISO/fbsd.iso \
  -boot order=cd,menu=on \
  -blockdev driver=file,aio=threads,node-name=imgleft,filename=../VM/left.img \
  -blockdev driver=raw,node-name=drive0,file=imgleft \
  -device virtio-blk-pci,drive=drive0,bootindex=1  \
  -netdev tap,id=nd0,ifname=tap0,script=no,downscript=no,br=bridge0 \
  -device e1000,netdev=nd0,mac=02:20:6c:65:66:74 \
  -name \"left\"
```



Save the above into a file (for example `left.sh`) and simply run: % `/bin/sh left.sh`

QEMU will start up a virtual machine in a separate window and boot the FreeBSD iso as shown in Figure 2. All command options such as `-cpu` and `-boot` are fully described in the QEMU man page qemu(1).



Figure 62. FreeBSD Boot Loader Menu



If the mouse is clicked in the QEMU console window, QEMU will "grab" the mouse as shown in Figure 4. Type `Ctl`+`Alt`+`G`" to release the mouse.

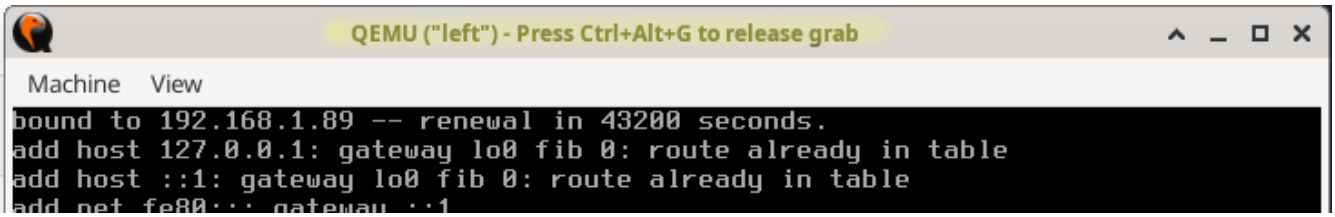


Figure 63. When QEMU Has Grabbed the Mouse



On FreeBSD, an initial QEMU installation can be somewhat slow. This is because the emulator writes filesystem formatting and metadata during the disk first use. Subsequent operations are generally much faster.

During the installation there are several points to note:

- Select to use UFS as the filesystem. ZFS does not perform well with small memory sizes.
- For networking use DHCP. If desired, configure IPv6 if supported by the local LAN.
- When adding the default user, ensure they are a member of the **wheel** group.

Once the installation completes, the virtual machine reboots into the newly installed FreeBSD image.

Login as `root` and update the system as follows:

```
# freebsd-update fetch
```

```
# freebsd-update install
# shutdown -r now
```



After a successful installation, QEMU will boot the operating system installed on the disk, and not the installation program.



QEMU supports a `-runas` option. For added security, include the option "-runas user_name" in the script listing above. See qemu(1) for details.

Login as `root` again and add any packages desired. To utilize the X Window system in the guest, see the section "Using the X Window System" below.

This completes the setup of the "left" VM.

To install the "right" VM, run the following script. This script has the modifications needed for tap1, format=qcow2, the image filename, the MAC address, and the terminal window name. If desired, include the "-runas" parameter as described in the above note.

```
/usr/local/bin/qemu-system-x86_64  -monitor none \
  -cpu qemu64 \
  -vga cirrus \
  -m 4096  -smp 4   \
  -cdrom ../ISO/fbsd.iso \
  -boot order=cd,menu=on \
  -drive
if=none,id=drive0,cache=writeback,aio=threads,format=qcow2,discard=unmap,file=../VM/ri
ght.qcow2 \
  -device virtio-blk-pci,drive=drive0,bootindex=1  \
  -netdev tap,id=nd0,ifname=tap1,script=no,downscript=no,br=bridge0 \
  -device e1000,netdev=nd0,mac=02:72:69:67:68:74 \
  -name \"right\"
```

Once the installation is complete, the "left" and "right" machines can communicate with each other and with the host. If there are strict firewall rules on the host, consider adding or modifying rules to allow the bridge and tap devices to communicate with each other.

24.6.3. Usage Tips

24.6.3.1. Using the X Window System

Installing Xorg describes how to set up the `X Window` system. Refer to that guide for initial `X Window` setup then consult Desktop Environments on how to set up a complete desktop.

This section demonstrates use of the XFCE desktop.

Once the installation is complete, login as a regular user, then type:

```
% startx
```

The XFCE4 window manager should start up and present a functioning graphical desktop as in Figure 5. On initial startup, it may take up to a minute to display the desktop. See the documentation at the XFCE website for usage details.

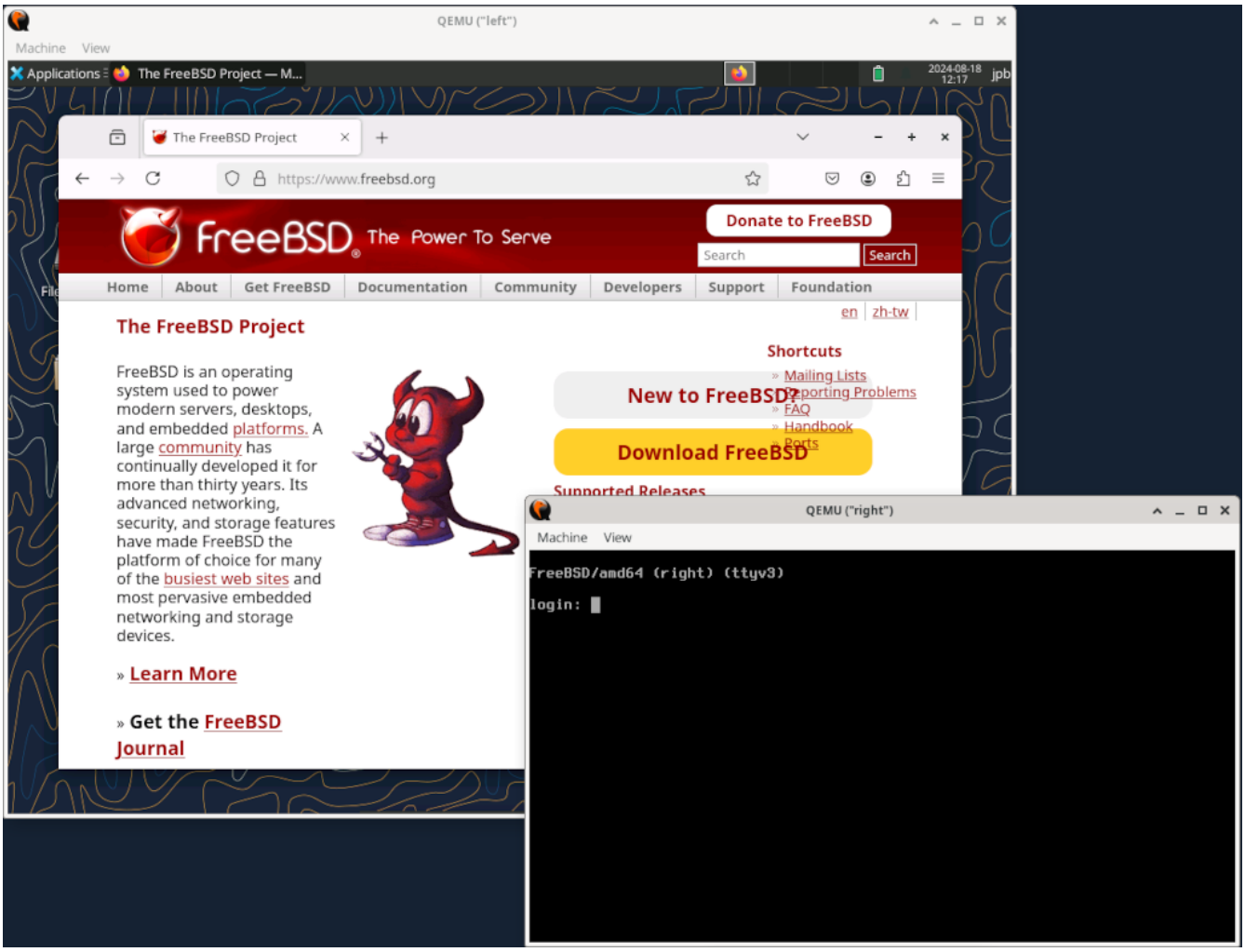


Figure 64. Both QEMU VMs



Adding more memory to the guest system may speed up the graphical user interface.

Here, the "left" VM has had the `X Window` system installed, while the "right" VM is still in text mode.

24.6.3.2. Using the QEMU Window

The QEMU window functions as a full FreeBSD console, and is capable of running multiple virtual terminals, just like a bare-metal system.

To switch to another virtual console, click into the QEMU window and type `Alt` + `F2` or `Alt` + `F3`. FreeBSD should switch to another virtual console. Figure 6 shows the "left" VM displaying the virtual console on `ttyv3`.

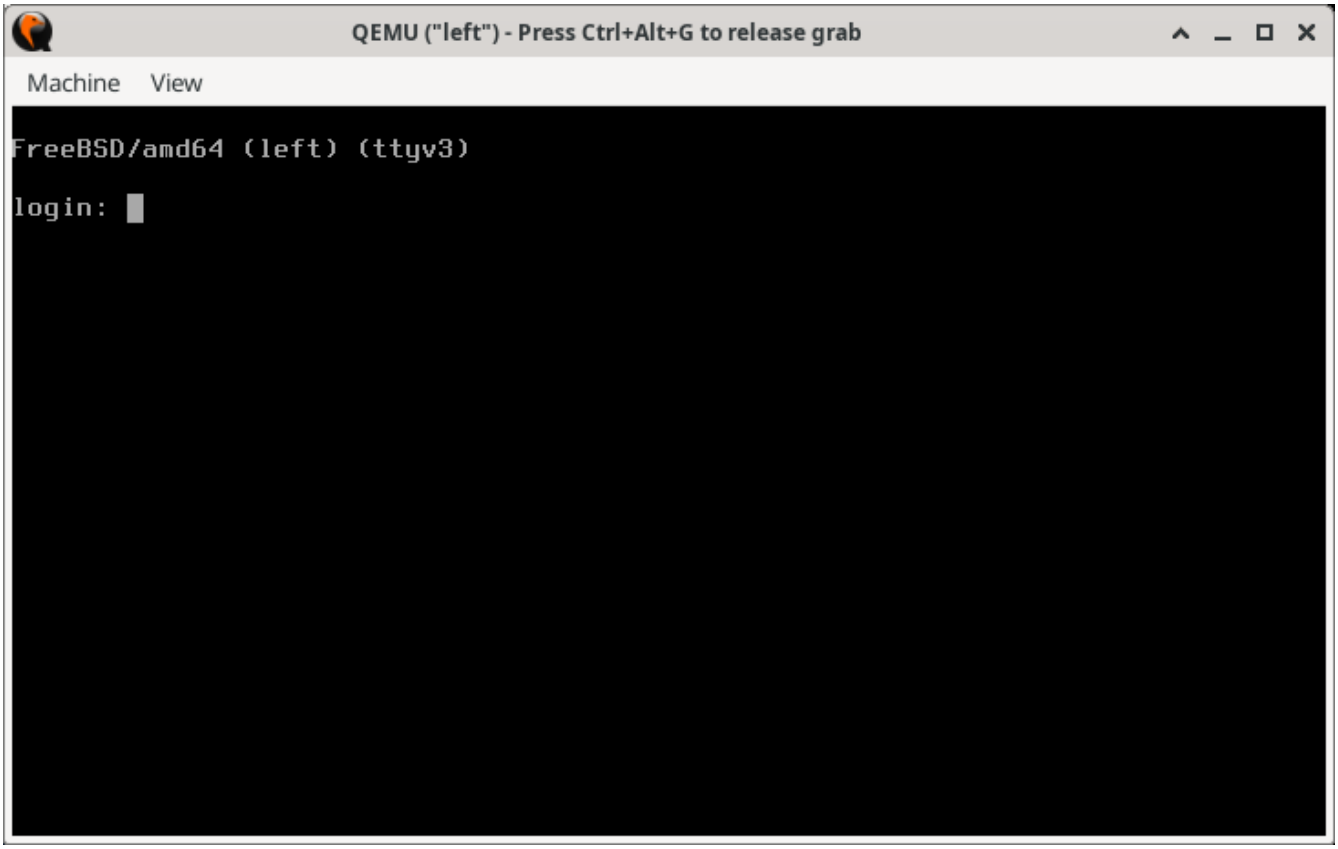


Figure 65. Switching to Another Virtual Console in the QEMU Window



The host current desktop manager or window manager may be already setup for another function with the `Alt` + `F1`, `Alt` + `F2` key sequences. If so, try typing `Ctl` + `Alt` + `F1`, `Ctl` + `Alt` + `F2`, or some other similar key combination. Check the window manager or desktop manager documentation for details.

24.6.3.3. Using the QEMU Window Menus

Another feature of the QEMU window is the `View` menu and the Zoom controls. The most useful is `Zoom to Fit`. When this menu selection is clicked, it is then possible to resize the QEMU window by clicking the window corner controls and resizing the window. Figure 7 shows the effect of resizing the "left" window while in graphics mode.

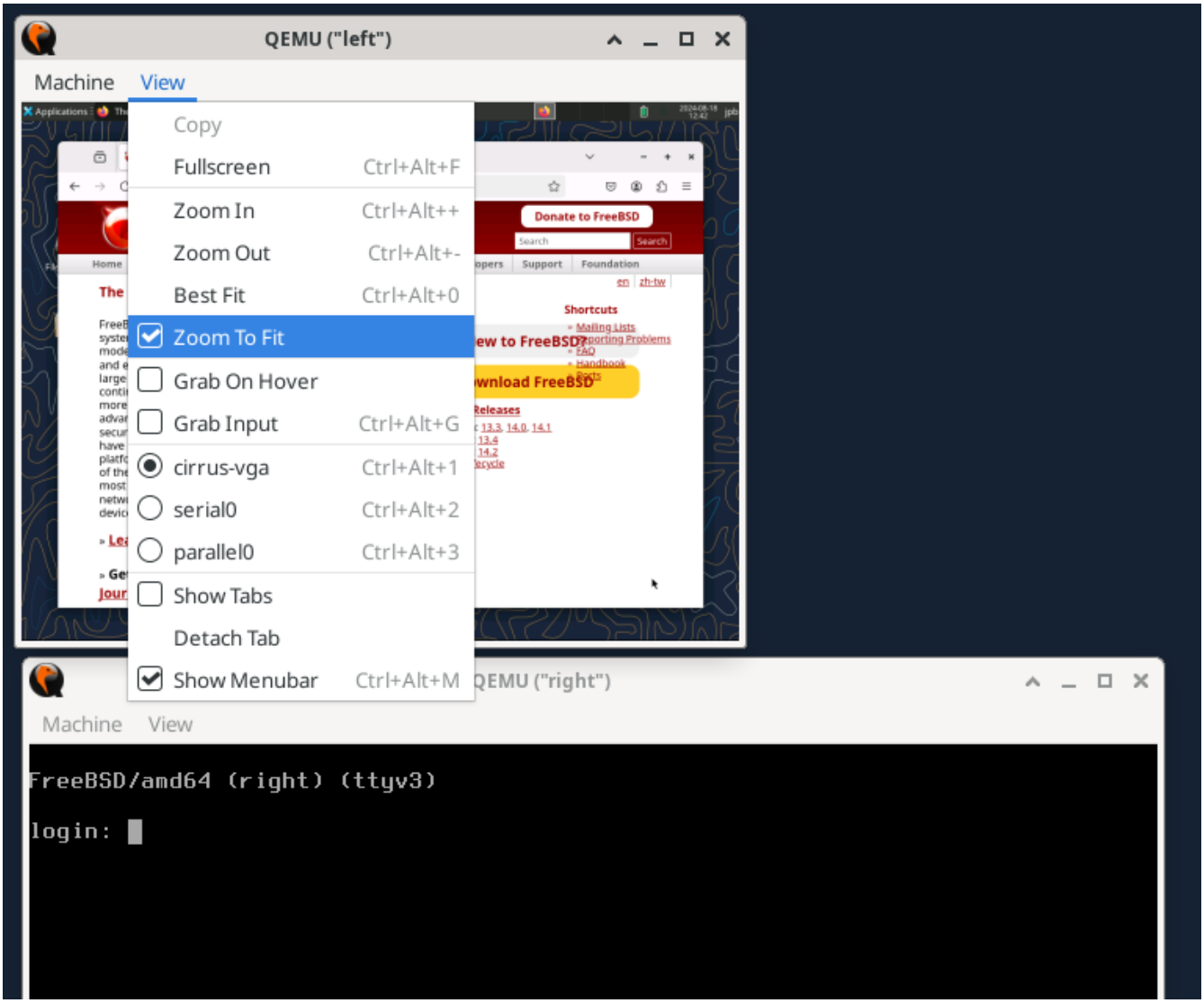


Figure 66. Using the View Menu `Zoom to Fit` Option

24.6.3.4. Other QEMU Window Menu Options

Also shown in the `View` menu are

- `cirrus-vga`, `serial0`, and `parallel0` options. These allow for switching input/output to the selected device.

The QEMU window `Machine` menu allows for four types of control over the guest VM:

- `Pause` allows for pausing the QEMU virtual machine. This may be helpful in freezing a fast scrolling window.
- `Reset` immediately resets the virtual machine back at cold "power on" state. As with a real machine, it is not recommended unless absolutely necessary.
- `Power Down` simulates an ACPI shutdown signal and the operating system goes through a graceful shutdown.
- `Quit` powers off the virtual machine immediately - also not recommended unless necessary.

24.6.4. Adding a Serial Port Interface to a Guest VM

To implement a serial console, a guest VM running FreeBSD needs to insert

```
console="comconsole"
```

in /boot/loader.conf to allow the use of the FreeBSD serial console.

The updated configuration below shows how to implement the serial console on the guest VM. Run the script to start the VM.

```
# left+serial.sh
echo
echo "NOTE: telnet startup server running on guest VM!"
echo "To start QEMU, start another session and telnet to localhost port 4410"
echo

/usr/local/bin/qemu-system-x86_64  -monitor none \
  -serial telnet:localhost:4410,server=on,wait=on\
  -cpu qemu64 \
  -vga std \
  -m 4096 \
  -smp 4   \
  -cdrom ../ISO/fbsd.iso \
  -boot order=cd,menu=on \
  -blockdev driver=file,aio=threads,node-name=imgleft,filename=../VM/left.img \
  -blockdev driver=raw,node-name=drive0,file=imgleft \
  -device virtio-blk-pci,drive=drive0,bootindex=1  \
  -netdev tap,id=nd0,ifname=tap0,script=no,downscript=no,br=bridge0 \
  -device e1000,netdev=nd0,mac=02:20:6c:65:66:74 \
  -name \"left\"
```

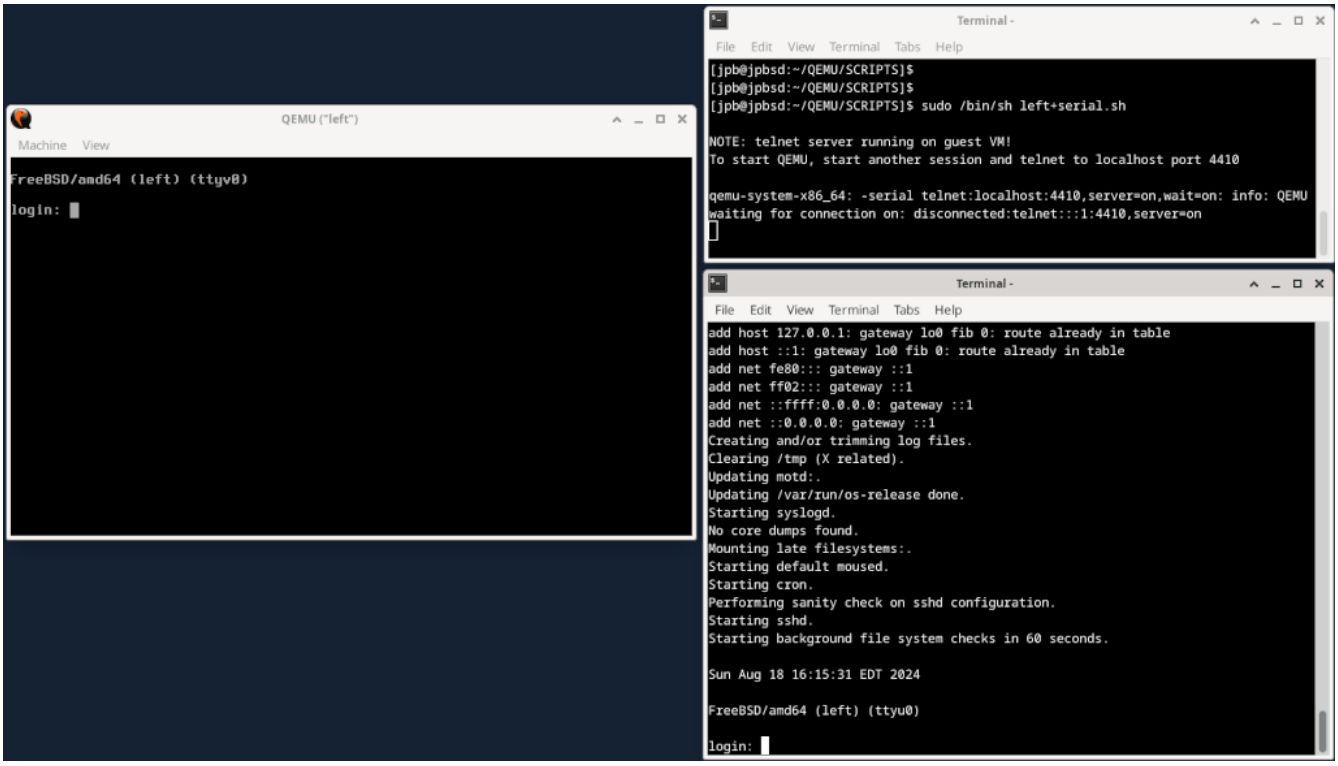


Figure 67. Enabling a Serial Port over TCP

In Figure 8, the serial port is redirected to a TCP port on the host system at VM startup and the QEMU monitor waits (`wait=on`) to activate the guest VM until a telnet(1) connection occurs on the indicated localhost port. After receiving a connection from a separate session, the FreeBSD system starts booting and looks for a console directive in /boot/loader.conf. With the directive "console=comconsole", FreeBSD starts up a console session on a serial port. The QEMU monitor detects this and directs the necessary character I/O on that serial port to the telnet session on the host. The system boots and once finished, login prompts are enabled on the serial port (`ttyu0`) and on the console (`ttyv0`).

It is important to note that the this serial redirect over TCP takes place outside the virtual machine. There is no interaction with any network on the virtual machine and therefore it is not subject to any firewall rules. Think of it like a dumb terminal sitting on an RS-232 or USB port on a real machine.

24.6.4.1. Notes on Using the Serial Console

On the serial console, if the window is resized, execute resizewin(1) to update the terminal size.

It may be desirable (even necessary) to stop syslog message from being sent to the console (both the QEMU console and the serial port). Consult syslog.conf(5) for details on redirecting console messages.



Once the /boot.loader.conf has been updated to permit a serial console, the guest VM will attempt to boot from the serial port every time. Ensure that the serial port is enabled as shown in the listing above, or update the /boot/loader.conf file to not require a serial console.

24.6.5. QEMU User Mode Emulation

QEMU also supports running applications that are precompiled on an architecture different from the host CPU. For example, it is possible to run a Sparc64 architecture operating system on an x86_64 host. This is demonstrated in the next section.

24.6.5.1. Setting up a SPARC64 Guest VM on an x86_64 Host

Setting up a new VM with an architecture different from the host involves several steps:

- Getting the software that will run on the guest VM
- Creating a new disk image for the guest VM
- Setting up a new QEMU script with the new architecture
- Performing the install

In the following procedure a copy of OpenBSD 6.8 SPARC64 software is used for this QEMU User Mode Emulation exercise.



Not all versions of OpenBSD Sparc64 work on QEMU. OpenBSD version 6.8 is known to work and was selected as the example for this section.

1. Download OpenBSD 6.8 Sparc64 from an OpenBSD archive.

   On the OpenBSD download sites, only the most current versions are maintained. It is necessary to search an archive to obtain past releases.

   ```
   % cd ~/QEMU/ISO
   % fetch https://mirror.planetunix.net/pub/OpenBSD-archive/6.8/sparc64/install68.iso
   ```

2. Creating a new disk image for the Sparc64 VM is similar to the "right" VM above. This case uses the QEMU qcow2 format for the disk:

   ```
   % cd ~/QEMU/VM
   qemu-img create -f qcow2 -o preallocation=full,lazy_refcounts=on sparc64.qcow2 16G
   ```

3. Use the script below for the new Sparc64 architecture. As with the above example, run the script, then start a new session and `telnet` to localhost on the port indicated:

   ```
   echo
   echo "NOTE: telnet startup server running on guest VM!"
   echo "To start QEMU, start another session and telnet to localhost port 4410"
   echo

   /usr/local/bin/qemu-system-sparc64 \
     -serial telnet:localhost:4410,server=on,wait=on \
     -machine sun4u,usb=off \
   ```

```
  -smp 1,sockets=1,cores=1,threads=1 \
  -rtc base=utc \
  -m 1024 \
  -boot d \
  -drive file=../VM/sparc64.qcow2,if=none,id=drive-ide0-0-1,format=qcow2,cache=none
\
  -cdrom ../ISO/install68.iso \
  -device ide-hd,bus=ide.0,unit=0,drive=drive-ide0-0-1,id=ide0-0-1 \
  -msg timestamp=on \
  -net nic,model=sunhme -net user \
  -nographic \
  -name \"sparc64\"
```

Note the following:

- The `-boot d` option boots from the QEMU CDROM device which is set as `-cdrom ../ISO/install68.iso`.
- As before, the `telnet` server option is set to wait for a separate connection on port 4410. Start up another session and use telnet(1) to connect to localhost on port 4410.
- The script sets the `-nographic` option meaning there is only serial port I/O. There is no graphical interface.
- Networking is not set up through the tap(4) / if_bridge(4) combination. This example uses a separate method of QEMU networking known as "Serial Line Internet Protocol" (SLIRP), sometimes referred to as "User Mode Networking". Documentation on this and other QEMU networking methods is here: QEMU Networking Documentation

If everything is set correctly, the system will boot as shown in Figure 9.



Figure 68. QEMU Booting OpenBSD 6.8 Sparc64 from CDROM During User Mode Emulation

Once the system is installed, modify the script and change the boot parameter to `-boot c`. This will indicate to QEMU to boot from the supplied hard disk, not the CDROM.

The installed system can be used just like any other guest virtual machine. However, the underlying architecture of the guest is Sparc64, not x86_64.



If the system is halted at the OpenBios console prompt `0 >`, enter `power-off` to exit the system.

Figure 10 shows a root login to the installed system and running uname(1).

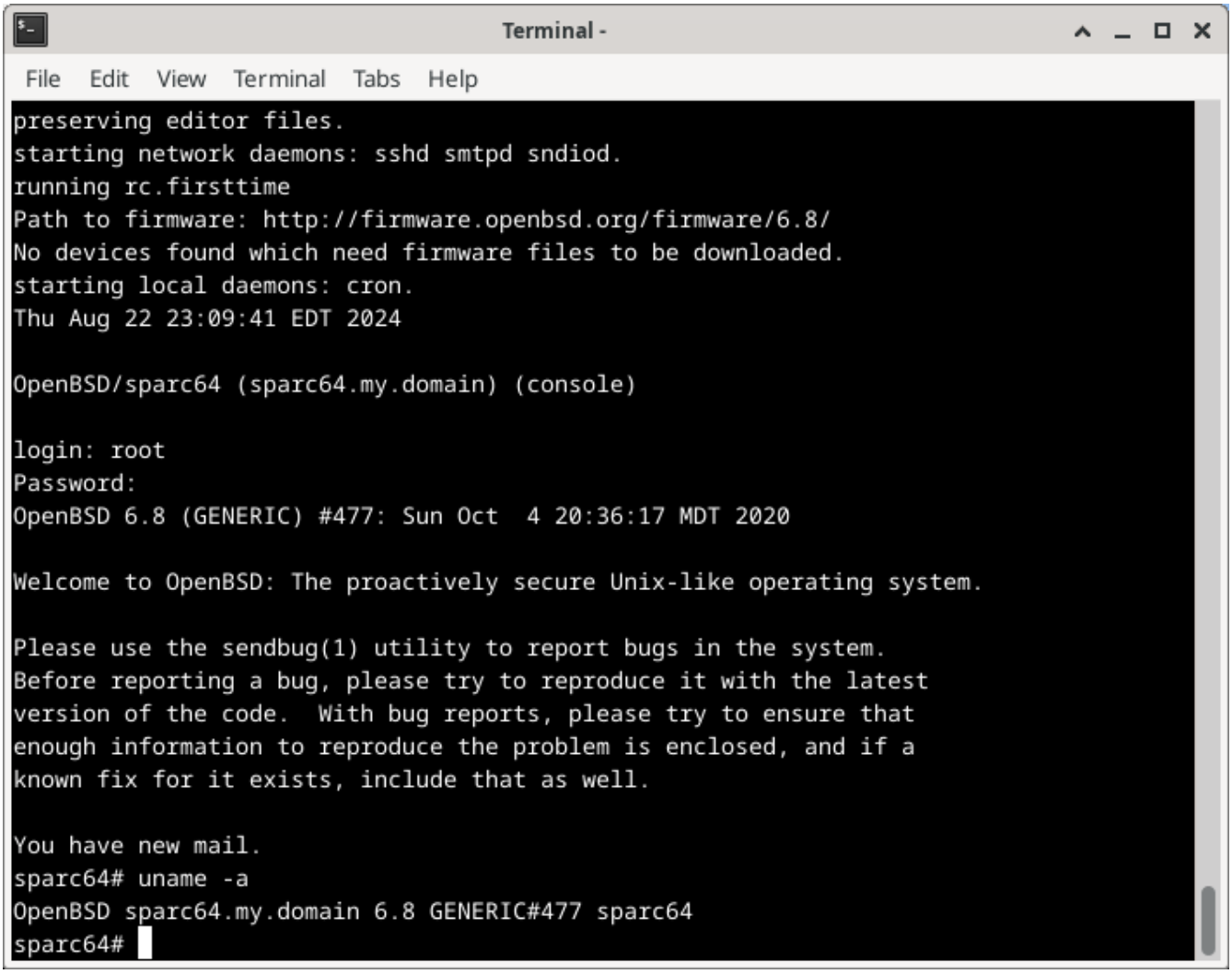


Figure 69. QEMU Booting from CDROM During User Mode Emulation

24.6.6. Using the QEMU Monitor

The QEMU monitor controls a running QEMU emulator (guest VM).

Using the monitor, it is possible to:

- Dynamically remove or insert devices, including disks, network interfaces, CD-ROMs, or floppies
- Freeze/unfreeze the guest VM, and save or restore its state from a disk file
- Gather information about the state of the VM and devices
- Change device settings on the fly

As well as many other operations.

The most common uses of the monitor are to examine the state of the VM, and to add, delete, or change devices. Some operations such as migrations are only available under hypervisor accelerators such as KVM, Xen, etc. and are not supported on FreeBSD hosts.

When using a graphical desktop environment, the simplest way to use the QEMU monitor is the `-monitor stdio` option when launching QEMU from a terminal session.